part1_source_cont...
Document

The uploaded part1_source_context.txt is the source-code context for
this experiment.

Analyze only the supplied source context.

Do not invent patterns or source locations.

Return the complete Part 1 pattern-discovery analysis requested in the prompt, including:
1. application architecture
2. confirmed patterns
3. rejected candidates
4. uncertain candidates
5. pattern count
6. pattern-to-source mapping
7. UML/diagram recommendations
8. CSV-ready pattern data

This response will be preserved verbatim as the raw LLM output for
an academic software-engineering case study. Do not omit your reasoning/evidence sections.

Worked for 56s

PART 1 — Pattern Discovery Analysis

Analysis basis: Only the supplied `part1_source_context.txt` was analyzed. No repository, Git history, external documentation, or web sources were used. Where the source context does not provide enough evidence to establish a pattern, the candidate is explicitly marked uncertain or rejected.

---

1. Application Architecture

1.1 Overall architecture

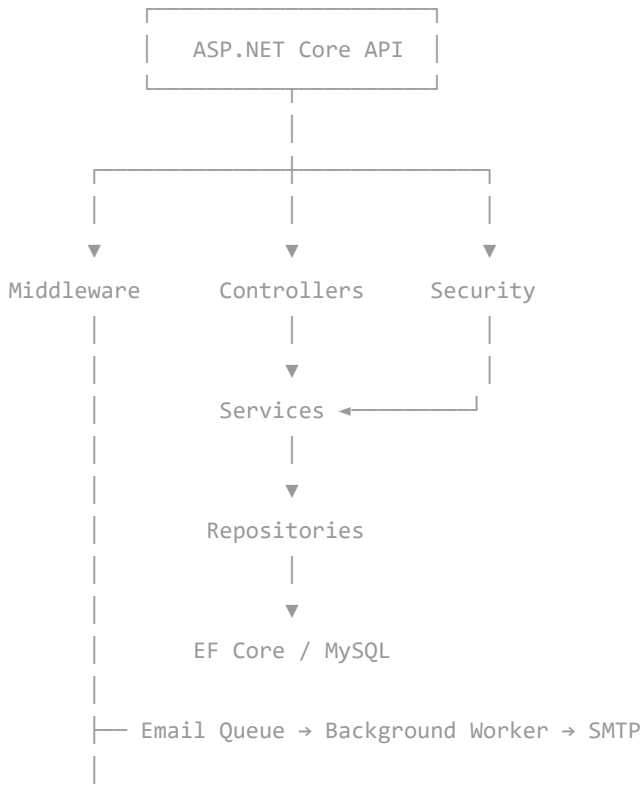
The supplied source context shows a **layered ASP.NET Core Web API architecture** organized primarily into:

Plain text

```
HTTP / Presentation Layer
        |
        ▼
Controllers
        |
        ▼
Service Layer
        |
        ▼
Repository Layer
        |
        ▼
EF Core / MySQL
```

There are also cross-cutting and integration components:

Plain text



The `Program.cs` configuration explicitly registers controllers, EF Core/MySQL, AutoMapper, authentication, services, repositories, email infrastructure, invoice-PDF services, and the Java microservice client. part1_source_context part1_source_context part1_source_context

1.2 Presentation / MVC-style Web API

The application uses ASP.NET Core controllers as its presentation boundary. For example, `VehicleController`, `BookingController`, and `AuthController` derive from `ControllerBase`, expose HTTP routes, and delegate business operations to service interfaces rather than implementing the main business logic themselves. part1_source_context

This is best characterized as an **ASP.NET Core MVC/Web API architectural style**, rather than a claim that the application implements a classical server-rendered MVC application.

1.3 Service Layer

Business operations are concentrated in services such as:

- `VehicleService`
- `BookingService`
- `EmailService`
- `JavaInvoicePdfService`
- `StaffService`
- `CustomerService`
- `LocationService`
- `AddonService`

The services are registered behind interfaces in the dependency-injection container. `BookingService`, for example, depends on repositories and other domain services rather than directly being invoked by the persistence layer. part1_source_context

1.4 Repository-based persistence

Persistence is isolated behind `IGenericRepository<TEntity,TKey>` and its `GenericRepository<TEntity,TKey>` implementation. The generic repository provides CRUD/query operations and delegates persistence to EF Core's `DbSet` and `DbContext`. part1_source_context part1_source_context

A specialized `IAvailabilityRepository` / `AvailabilityRepository` exists for queries that do not fit the generic CRUD abstraction. part1_source_context

1.5 Cross-cutting middleware

The HTTP pipeline contains:

```
Plain text

ExceptionHandlingMiddleware
        ↓
RequestLoggingMiddleware
        ↓
```

```
CORS
        ↓
Authentication
        ↓
Authorization
        ↓
Controllers
```

The middleware components receive a `RequestDelegate` and invoke `_next(context)`, providing explicit pipeline behavior around request processing. part1_source_context

1.6 External/legacy integration

The application retains integration with a Java Spring Boot backend through:

```
LegacyProxyController
        ↓
IJavaMicroserviceClient
        ↓
JavaMicroserviceClient
        ↓
HttpClient
        ↓
Java backend
```

The source explicitly describes this as a **strangler-pattern integration**. part1_source_context

---

2. Confirmed Patterns

The following patterns have sufficient evidence in the supplied source context.

Pattern 1 — Repository Pattern

Status: CONFIRMED

Evidence

`IGenericRepository<TEntity,TKey>` defines a reusable persistence abstraction containing:

- `GetByIdAsync`
- `GetAllAsync`
- `FindAsync`
- `FirstOrDefaultAsync`
- `ExistsAsync`
- `AddAsync`

- `Update`
- `Remove`
- `SaveChangesAsync`

`GenericRepository<TEntity,TKey>` implements the abstraction using EF Core. The source explicitly states that it is reused by entities that do not require custom queries. part1_source_context

The repository implementation encapsulates `FlemanDbContext` and `DbSet<TEntity>`.

part1_source_context

Pattern structure

```
IGenericRepository<TEntity,TKey>
              ▲
              | implements
              |
GenericRepository<TEntity,TKey>
              |
              ▼
       FlemanDbContext
              |
              ▼
          DbSet<TEntity>
```

Confidence

High.

This is one of the strongest pattern identifications in the source.

Pattern 2 — Dependency Injection / Inversion of Control

Status: CONFIRMED

Evidence

`Program.cs` registers abstractions against concrete implementations:

```
IGenericRepository → GenericRepository
IVehicleService → VehicleService
IBookingService → BookingService
IEmailService → EmailService
```

```
IInvoicePdfService → JavaInvoicePdfService
IJavaMicroserviceClient → JavaMicroserviceClient
```

The source contains explicit scoped, singleton, hosted-service, and typed- `HttpClient` registrations.

part1_source_context

Classes receive dependencies through constructors. For example, `VehicleService` receives repositories, an availability repository, and `IMapper`. part1_source_context

`BookingService` has an even broader constructor dependency graph containing repositories, domain services, authentication access, email service, and logging. part1_source_context

Confidence

High.

This is a concrete implementation of dependency injection and IoC through the ASP.NET Core DI container.

Pattern 3 — Service Layer

Status: CONFIRMED

Evidence

The controllers delegate application operations to service interfaces.

For example, `BookingController` invokes:

Plain text

```
IBookingService.CreateBookingAsync()
IBookingService.GetBookingByConfirmationAsync()
IBookingService.ModifyBookingAsync()
IBookingService.CancelBookingAsync()
```

rather than implementing booking business rules directly. part1_source_context

`BookingService` contains the booking workflow, including customer resolution, availability checking, booking creation, pricing, add-ons, email notification, cancellation, and access validation.

part1_source_context

Architectural relationship

```
BookingController
        │
        ▼
IBookingService
        │
        ▼
BookingService
        │
        ├── repositories
        ├── availability service
        ├── customer service
        ├── vehicle service
        └── email service
```

Confidence

High.

Pattern 4 — Middleware Pipeline / Chain of Responsibility

Status: CONFIRMED

Evidence

`ExceptionHandlingMiddleware` receives a `RequestDelegate` and passes control onward using `_next(context)`. part1_source_context

`RequestLoggingMiddleware` has the same structure: it performs work before the downstream component, calls `_next(context)`, and performs logging afterward in `finally`. part1_source_context

`Program.cs` composes both middleware components into the HTTP pipeline. part1_source_context

Why this qualifies

Each middleware component:

1. receives the request,
2. performs its own responsibility,
3. forwards the request to the next component,
4. may perform additional processing afterward.

That is the essential structural behavior of a **Chain of Responsibility/pipeline**.

Confidence

High, with the terminology caveat that ASP.NET Core normally calls this the **middleware pipeline**. "Chain of Responsibility" is the corresponding design-pattern interpretation.

Pattern 5 — Producer–Consumer / Asynchronous Work Queue

Status: CONFIRMED

Evidence

`EmailBackgroundQueue` uses a bounded `Channel<Func<IEmailSender,...>>` with capacity 100. Producers enqueue work through `Enqueue()`, while consumers retrieve work through `ReadAllAsync()`. part1_source_context

`EmailService` does not perform the SMTP operation inline. Instead it places an operation into the queue. part1_source_context

`EmailQueueHostedService` continuously consumes the queued work and executes it using `IEmailSender`. part1_source_context

Structure

```
Plain text
EmailService
    |
    | Enqueue(work)
    ▼
Bounded Channel
    |
    | ReadAllAsync()
    ▼
EmailQueueHostedService
    |
    ▼
IEmailSender
    |
    ▼
MailKit SMTP
```

Confidence

High.

This is more precisely a concurrency/application-infrastructure pattern than a GoF object-creation/structural pattern.

Pattern 6 — Adapter Pattern

Status: CONFIRMED

Evidence

`IJavaMicroserviceClient` provides an application-facing abstraction for communication with the Java backend. `JavaMicroserviceClient` implements that abstraction while internally using `HttpClient`. part1_source_context

It also performs compatibility transformations between the .NET and Java representations, including a custom `DateTime` converter and Java-specific JSON serialization options. part1_source_context

Why this qualifies

The application does not expose `HttpClient` or Java-specific communication details directly to its callers. Instead:

```
Application
    │
    ▼
IJavaMicroserviceClient
    │
    ▼
JavaMicroserviceClient
    │
    ├── JSON adaptation
    ├── DateTime adaptation
    └── HTTP communication
          │
          ▼
     Java backend
```

Confidence

High.

The interface-plus-wrapper structure and explicit representation conversion provide strong evidence.

Pattern 7 — Proxy Pattern

Status: CONFIRMED

Evidence

`LegacyProxyController` exposes `/api/legacy/...` endpoints and forwards the calls to `IJavaMicroserviceClient`. It does not implement the underlying Java service itself.

part1_source_context

For example:

```
Plain text
GET /api/legacy/states
        |
        ▼
LegacyProxyController
        |
        ▼
IJavaMicroserviceClient
        |
        ▼
Java backend
```

Confidence

High.

The class is explicitly named `LegacyProxyController`, and its behavior is directly forwarding requests.

Pattern 8 — Strangler Fig Pattern

Status: CONFIRMED

Evidence

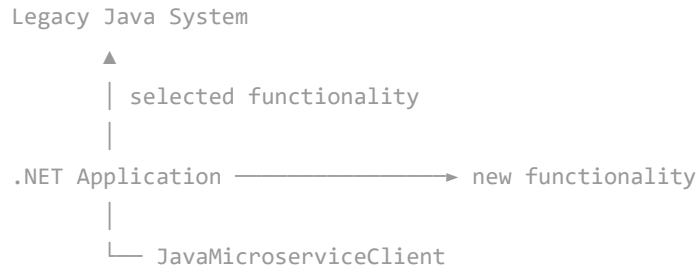
The source explicitly describes `IJavaMicroserviceClient` as demonstrating a **strangler-pattern integration**. It also states that the rest of the API is self-sufficient and that the Java integration is opt-in rather than a hard runtime dependency. part1_source_context

The invoice PDF functionality similarly allows the active implementation to use the Java backend while retaining a local `InvoicePdfService` implementation as an alternative. `Program.cs` currently registers `JavaInvoicePdfService`. part1_source_context

Interpretation

The .NET application is progressively taking over functionality while retaining selected legacy Java functionality through an integration boundary.

```
Plain text
```



Confidence

High, because the source itself calls this a strangler-pattern integration.

Pattern 9 — Data Transfer Object (DTO) Pattern

Status: CONFIRMED

Evidence

The service layer operates extensively with DTOs rather than exposing persistence entities directly.

For example:

```
Plain text
Car → CarDTO
CarType → CarTypeDTO
BookingHeader → BookingResponseDTO
BookingDetail → BookingDetailDTO
```

`VehicleService` explicitly maps domain entities into DTOs, while `BookingService` constructs a `BookingResponseDTO` and populates nested DTO structures. part1_source_context part1_source_context

Controllers also receive request DTOs such as `CreateBookingRequest`, `ModifyBookingRequest`, `LoginRequest`, and `RegisterRequest`. part1_source_context

Confidence

High.

The DTO pattern is directly visible in the public service/controller boundaries.

---

3. Confirmed Pattern Count

For this analysis, the **confirmed pattern count is 9**:

#	Pattern	Category	Confidence
1	Repository	Enterprise/application	High
2	Dependency Injection / IoC	Architectural/design	High
3	Service Layer	Enterprise/application	High
4	Middleware Pipeline / Chain of Responsibility	Architectural/design	High
5	Producer–Consumer	Concurrency/infrastructure	High
6	Adapter	Structural/integration	High
7	Proxy	Structural/integration	High
8	Strangler Fig	System modernization	High
9	DTO	Enterprise/application	High

Important scope note: MVC/Web API and layered architecture are reported under **Application Architecture** rather than counted in the nine pattern count. This avoids mixing architectural styles with implementation-level design patterns.

---

4. Rejected Pattern Candidates

The following patterns were considered but **should not be reported as confirmed patterns** based on this source context.

4.1 Factory Pattern — REJECTED

There are many calls such as:

```
Plain text
WebApplication.CreateBuilder(...)
builder.Build()
AddScoped(...)
AddSingleton(...)
AddHttpClient(...)
```

However, these are primarily ASP.NET Core framework APIs and dependency-container configuration.

There is no application-authored factory class that encapsulates creation of a family of objects or selects concrete products.

Decision: REJECT.

The presence of framework factory mechanisms is insufficient evidence for an application-level Factory pattern. The application configuration is shown in `Program.cs`. part1_source_context

part1_source_context

4.2 Singleton Pattern — REJECTED as a design-pattern claim

`Program.cs` uses:

```
Plain text
AddSingleton<IEmailBackgroundQueue, EmailBackgroundQueue>()
AddSingleton<IEmailSender, MailKitEmailSender>()
```

But an ASP.NET Core **singleton service lifetime** does not by itself establish that the GoF Singleton pattern has been implemented.

There is no explicit private constructor/static instance/accessor structure.

Decision: REJECT as an authored Singleton design pattern.

The registrations should instead be documented as **DI lifetime configuration**. part1_source_context

---

4.3 Builder Pattern — REJECTED

`WebApplication.CreateBuilder()` and fluent ASP.NET Core configuration may superficially resemble Builder usage.

However, this is framework-provided API usage rather than a custom Builder implementation in the supplied application.

Decision: REJECT as an application pattern.

4.4 Facade Pattern — REJECTED

`EmailService` provides a simpler interface over queue-based email sending, and services such as `BookingService` coordinate several dependencies.

However, the source does not establish a deliberate Facade abstraction whose primary purpose is to provide a unified simplified interface to a complex subsystem.

Calling this a Facade would therefore be interpretive rather than source-confirmed.

Decision: REJECT.

4.5 Decorator Pattern — REJECTED

Middleware wraps downstream request processing, but the source demonstrates the ASP.NET Core middleware pipeline rather than decorators that dynamically add behavior to an object implementing a component interface.

Decision: REJECT as Decorator; classify the mechanism as Middleware Pipeline / Chain of Responsibility instead.

4.6 Observer Pattern — REJECTED

There are authentication event callbacks such as `OnCreatingTicket` and `OnRedirectToAuthorizationEndpoint`, but these are framework event hooks.

The source does not show an application-authored subject maintaining a collection of observers/subscribers.

Decision: REJECT.

The callbacks are part of Google authentication configuration. part1_source_context

---

4.7 Command Pattern — REJECTED

The email queue stores:

```
Plain text
Func<IEmailSender, CancellationToken, Task>
```

These queued delegates are **command-like work items**, but there is no explicit command abstraction such as `ICommand`, concrete command objects, or a command invoker.

The stronger and more defensible classification is **Producer–Consumer / Work Queue**.

Decision: REJECT Command as a named design pattern.

4.8 Template Method Pattern — REJECTED

There is no clear base class containing an invariant algorithm with overridable steps in the supplied source.

Decision: REJECT.

4.9 State Pattern — REJECTED

The application contains states such as:

```
Plain text
BookingStatus.CONFIRMED
BookingStatus.ONGOING
BookingStatus.CANCELLED
CarStatus.AVAILABLE
CarStatus.BOOKED
CarStatus.UNDER_MAINTENANCE
```

However, these are enums and conditional business rules, not polymorphic state objects.

Decision: REJECT.

---

5. Uncertain Candidates

These candidates have some structural evidence but not enough to label them as confirmed.

5.1 Strategy Pattern — UNCERTAIN / STRATEGY-CAPABLE

There is a particularly interesting case around invoice PDF generation.

The interface is:

```
Plain text
IInvoicePdfService
```

The active implementation is:

```
Plain text
```

```
JavaInvoicePdfService
```

while a local:

```
Plain text
InvoicePdfService
```

implementation exists in the source context but is commented out. `Program.cs` explicitly states that the registration can be swapped back to `InvoicePdfService` to generate PDFs locally.

part1_source_context

`JavaInvoicePdfService` implements the common `IInvoicePdfService` abstraction.

part1_source_context

This resembles Strategy:

```
Plain text
                IInvoicePdfService
                        ▲
                        |
            ┌───────────┴──────────┐
            |                      |
JavaInvoicePdfService    InvoicePdfService
       active                 alternative
```

However, the alternative implementation is commented out and there is no runtime strategy-selection mechanism.

Decision: UNCERTAIN / strategy-capable, not counted among the nine confirmed patterns.

5.2 Data Mapper — UNCERTAIN as an explicit authored pattern

The application uses AutoMapper:

```
Plain text
builder.Services.AddAutoMapper(...)
```

and services call `_mapper.Map<...>()`. part1_source_context part1_source_context

This clearly provides **entity-to-DTO mapping**, but the supplied context does not show custom AutoMapper profile implementations.

Therefore, it is safe to state:

> AutoMapper-based mapping is confirmed, but an independently authored Data Mapper pattern should not be claimed without additional source evidence.

Decision: UNCERTAIN as a separate pattern; DTO pattern is confirmed.

5.3 Unit of Work — UNCERTAIN / framework-backed

`GenericRepository` exposes:

```
Plain text
SaveChangesAsync()
```

and ultimately calls:

```
Plain text
Context.SaveChangesAsync()
```

This can resemble Unit of Work behavior because EF Core's `DbContext` coordinates persistence.

part1_source_context

However, the source does not define an application-level `IUnitOfWork` abstraction.

Decision: UNCERTAIN; do not count as a custom Unit of Work pattern.

---

6. Pattern-to-Source Mapping

Pattern	Primary source	Supporting source	Evidence
Repository	`Repository/IGenericRepository.cs`	`Repository/GenericRepository.cs`	Generic CRUD abstraction and EF implementation
Dependency Injection / IoC	`Program.cs`	All service/controller constructors	Interface-to-implementation registration and constructor injection

Pattern	Primary source	Supporting source	Evidence
Service Layer	`Service/*Service.cs`	`Controllers/*Controller.cs`	Controllers delegate business operations to services
Middleware / Chain of Responsibility	`Middleware/ExceptionHandlingMiddleware.cs`	`Middleware/RequestLoggingMiddleware.cs` , `Program.cs`	`_next(context)` pipeline
Producer–Consumer	`Service/EmailBackgroundQueue.cs`	`EmailService.cs` , `EmailQueueHostedService.cs`	Bounded channel + producer + background consumer
Adapter	`Service/IJavaMicroserviceClient.cs`	`Service/JavaMicroserviceClient.cs`	Java-facing abstraction + HTTP/JSON/DateTime adaptation
Proxy	`Controllers/LegacyProxyController.cs`	`IJavaMicroserviceClient.cs`	`/api/legacy` forwards to Java service
Strangler Fig	`Service/IJavaMicroserviceClient.cs`	`JavaInvoicePdfService.cs` , `Program.cs`	Explicitly described as strangler integration
DTO	`Controllers/*` , `Service/*`	`BookingService.cs` , `VehicleService.cs`	Request/response DTO boundary and entity-to-DTO transformation

Particularly strong source mappings

The repository structure is directly visible in the generic repository interface and implementation.

part1_source_context

The service/controller separation is visible in the vehicle and booking paths. part1_source_context

part1_source_context

The middleware chain is explicit in both middleware implementations and `Program.cs` .

part1_source_context part1_source_context

The asynchronous email queue is represented by three cooperating components. part1_source_context

The legacy integration is represented by the Java client and proxy controller. part1_source_context

7. UML / Diagram Recommendations

The source context supports several useful diagrams for the academic case study.

7.1 Recommended Diagram 1 — Overall Architecture Diagram

Highest priority.

Show:

```
Client
  │
  ▼
ASP.NET Core Controllers
  │
  ▼
Service Layer
  │
  ├──────────────┐
  ▼              ▼
Repositories   Other Services
  │              │
  ▼              ├── Email Queue
EF Core          ├── PDF Service
  │              └── Java Client
  ▼
MySQL
```

Also show the middleware pipeline before the controllers.

This diagram demonstrates the **layered architecture + DI + repository + service layer** relationship.

7.2 Recommended Diagram 2 — Repository Pattern Class Diagram

```
          <<interface>>
      IGenericRepository<TEntity,TKey>
                ▲
                │
                │ implements
                │
     GenericRepository<TEntity,TKey>
                │
                ▼
          FlemanDbContext
```



Then separately show:

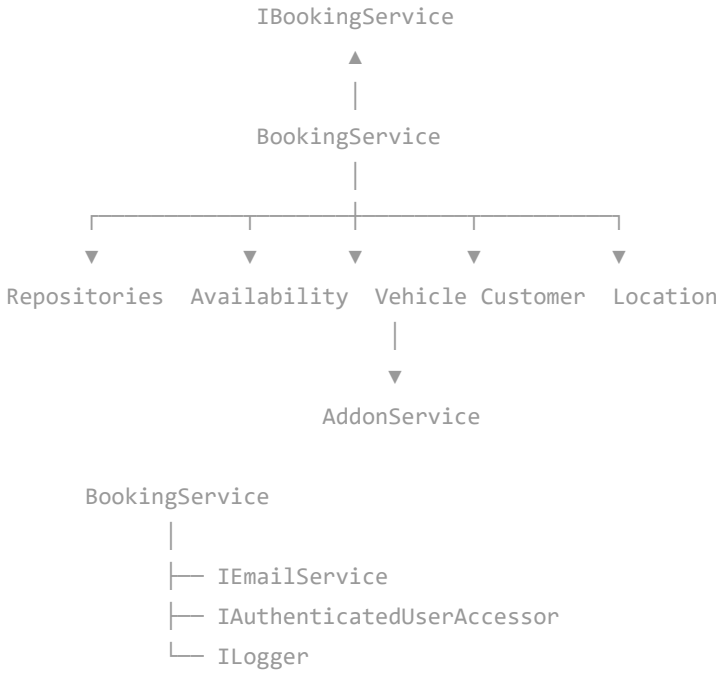
```
IAvailabilityRepository
          ▲
          |
AvailabilityRepository
          |
          ▼
   FlemanDbContext
```

This is strongly supported by the source. part1_source_context

---

7.3 Recommended Diagram 3 — Booking Service Dependency Diagram

This would be particularly useful because `BookingService` is one of the most dependency-rich components.



The constructor evidence is explicit. part1_source_context

7.4 Recommended Diagram 4 — Middleware Chain

Plain text

```
HTTP Request
     |
     ▼
ExceptionHandlingMiddleware
     |
     ▼
RequestLoggingMiddleware
     |
     ▼
CORS
     |
     ▼
Authentication
     |
     ▼
Authorization
     |
     ▼
Controller
```

This is ideal for demonstrating the Chain of Responsibility/pipeline interpretation. part1_source_context

---

7.5 Recommended Diagram 5 — Email Producer–Consumer

Plain text

```
Booking / Other Service
          |
          ▼
     EmailService
          |
          | Enqueue
          ▼
  Bounded Channel<Work>
          |
          | ReadAllAsync
          ▼
EmailQueueHostedService
          |
          ▼
     IEmailSender
          |
          ▼
```

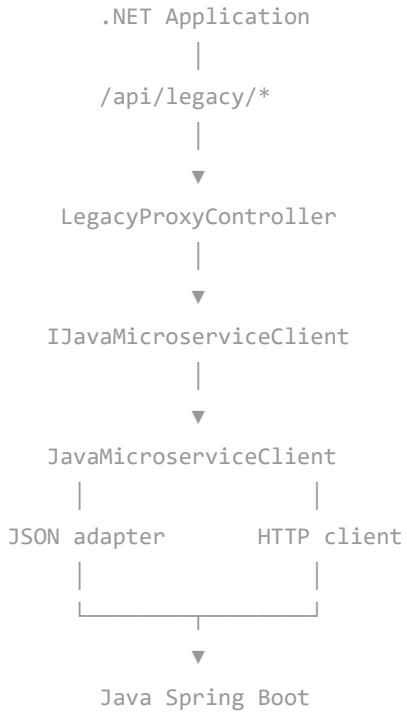
```
MailKitEmailSender
        |
```

This is one of the clearest pattern diagrams in the source. part1_source_context

---

7.6 Recommended Diagram 6 — Legacy Strangler/Adapter/Proxy Integration

Plain text



This diagram can simultaneously illustrate:

- Proxy
- Adapter
- Strangler Fig integration

The source explicitly identifies the Java integration as a strangler-pattern integration.

part1_source_context

---

8. Evidence-Based Pattern Reasoning Summary

The strongest evidence is not simply the presence of similarly named classes. The following behavioral structures are what justify the classifications:

Repository

An abstraction explicitly separates persistence operations from services, and a reusable implementation encapsulates EF Core. part1_source_context

Service Layer

Controllers invoke services while substantial application logic resides in those services.

part1_source_context part1_source_context

Dependency Injection

Concrete implementations are bound to interfaces centrally and dependencies are supplied through constructors. part1_source_context

Middleware / Chain

Components receive `_next` and explicitly forward control through the request pipeline.

part1_source_context

Producer–Consumer

One component enqueues email work while a separate background worker consumes and executes it.

part1_source_context

Adapter

`JavaMicroserviceClient` hides HTTP and Java-specific serialization compatibility behind an application interface. part1_source_context

Proxy

`LegacyProxyController` directly forwards requests to the external service through the client abstraction. part1_source_context

Strangler Fig

The source explicitly identifies the legacy integration as a strangler-pattern integration and retains only selected legacy functionality. part1_source_context

DTO

Requests and responses cross the controller/service boundary through DTO types, while services construct nested response DTOs rather than exposing database objects directly. part1_source_context

---

9. CSV-Ready Pattern Data

The following is deliberately formatted as CSV-style data so it can be copied into a spreadsheet or saved as `.csv`.

CSV

```csv
Pattern,Category,Status,Confidence,Primary_Source,Supporting_Source,Evidence,Notes
Repository,Enterprise/Application,Confirmed,High,Repository/IGenericRepository.cs,Repository/GenericRepos
Dependency Injection / IoC,Architectural/Design,Confirmed,High,Program.cs,Service and Controller construc
Service Layer,Enterprise/Application,Confirmed,High,Service/*Service.cs,Controllers/*Controller.cs,"Contr
Middleware Pipeline / Chain of Responsibility,Architectural/Design,Confirmed,High,Middleware/ExceptionHar
Producer-Consumer,Concurrency/Infrastructure,Confirmed,High,Service/EmailBackgroundQueue.cs,Service/Emai
Adapter,Structural/Integration,Confirmed,High,Service/IJavaMicroserviceClient.cs,Service/JavaMicroservice
Proxy,Structural/Integration,Confirmed,High,Controllers/LegacyProxyController.cs,Service/IJavaMicroservic
Strangler Fig,System Modernization,Confirmed,High,Service/IJavaMicroserviceClient.cs,Service/JavaInvoiceP
DTO,Enterprise/Application,Confirmed,High,Controllers/*Controller.cs,Service/BookingService.cs; Service/V
Strategy,Behavioral/Design,Uncertain,Medium,Service/IInvoicePdfService.cs,Service/JavaInvoicePdfService.c
Data Mapper,Enterprise/Application,Uncertain,Medium,Program.cs,Service/VehicleService.cs,"AutoMapper maps
Unit of Work,Enterprise/Application,Uncertain,Low,Repository/GenericRepository.cs,Program.cs,"DbContext S
Factory,Creational/Design,Rejected,High,Program.cs,,"Framework creation/configuration APIs are present bu
Singleton,Creational/Design,Rejected,High,Program.cs,,"AddSingleton configures DI lifetime but no explici
Builder,Creational/Design,Rejected,High,Program.cs,,"WebApplication.CreateBuilder is framework API usage
Facade,Structural/Design,Rejected,Medium,Service/EmailService.cs,,"Some services simplify other component
Decorator,Structural/Design,Rejected,High,Middleware/*Middleware.cs,,"Middleware wraps request processing
Observer,Behavioral/Design,Rejected,High,Program.cs,,"Authentication callbacks are framework hooks, not a
Command,Behavioral/Design,Rejected,Medium,Service/EmailBackgroundQueue.cs,"Queued delegates are command
```

---

10. Final Part 1 Finding

Confirmed patterns

9 confirmed patterns

1. **Repository**
2. **Dependency Injection / IoC**
3. **Service Layer**
4. **Middleware Pipeline / Chain of Responsibility**
5. **Producer–Consumer**
6. **Adapter**
7. **Proxy**
8. **Strangler Fig**
9. **DTO**

Uncertain candidates

1. **Strategy** — supported by the `IInvoicePdfService` abstraction and alternative PDF implementations, but runtime strategy selection is not demonstrated.
2. **Data Mapper** — AutoMapper is used, but the supplied context does not establish a custom Data Mapper implementation.
3. **Unit of Work** — EF Core `DbContext` provides similar coordination, but no explicit application-level Unit of Work abstraction is shown.

Rejected candidates

- Factory
- Singleton as an authored GoF implementation
- Builder as an application-authored pattern
- Facade
- Decorator
- Observer
- Command
- Template Method
- State

Architecture conclusion

The source context supports describing the application as a **layered ASP.NET Core Web API/MVC-style application with Controller → Service → Repository separation, dependency injection, EF Core persistence, middleware-based cross-cutting concerns, asynchronous email processing, and legacy Java integration through an adapter/proxy boundary as part of a strangler-style modernization approach**. part1_source_context part1_source_context

This classification deliberately avoids treating every framework feature, interface, enum, or fluent API call as a design pattern.